



Builder Design Pattern — Complete Guide

1. What is the Builder Design Pattern?

The **Builder pattern** is a **creational design pattern** that provides a flexible way to construct **complex objects step by step**, without exposing the construction logic to the client.

Instead of having constructors with long parameter lists (**telescoping constructors**), Builder lets us build objects gradually and in a readable way.

2. Why Builder? (The Need)

Problem Example (Telescoping Constructors)

```
public class User {
    private final String firstName;
    private final String lastName;
    private final int age;
    private final String phone;
    private final String address;

    public User(String firstName, String lastName, int age, String phone,
String address) {
        this.firstName = firstName;
        this.lastName = lastName;
        this.age = age;
        this.phone = phone;
        this.address = address;
    }
}
```

- What if there are 10+ parameters?
- What if most are optional?
- This leads to **hard-to-read constructors** and multiple overloads.

✓ Builder solves this by: - Making construction **step-by-step** - Allowing **optional parameters easily** - Making objects **immutable** while still customizable

3. Builder Structure

- **Product** → The complex object being built.
- **Builder** → Abstract interface for building steps.
- **Concrete Builder** → Implements building logic.
- **Director (optional)** → Controls construction order.
- **Client** → Uses the builder.

4. Java Implementation Example

With Builder

```
public class User {
    private final String firstName;
    private final String lastName;
    private final int age;
    private final String phone;
    private final String address;

    private User(Builder builder) {
        this.firstName = builder.firstName;
        this.lastName = builder.lastName;
        this.age = builder.age;
        this.phone = builder.phone;
        this.address = builder.address;
    }

    public static class Builder {
        private final String firstName;
        private final String lastName;
        private int age;
        private String phone;
        private String address;

        public Builder(String firstName, String lastName) {
            this.firstName = firstName;
            this.lastName = lastName;
        }

        public Builder age(int age) { this.age = age; return this; }
        public Builder phone(String phone) { this.phone = phone; return
this; }
        public Builder address(String address) { this.address = address;
return this; }

        public User build() { return new User(this); }
    }

    @Override
    public String toString() {
        return String.format("User: %s %s, Age: %d, Phone: %s, Address: %s",
            firstName, lastName, age, phone, address);
    }
}
```

Usage:

```
User user = new User.Builder("Hardik", "Singh")
    .age(25)
    .phone("9999999999")
    .address("India")
    .build();
System.out.println(user);
```

5. Why is Builder Better?

Problem	Solution with Builder
Too many constructor parameters	Chainable, step-by-step building
Hard to read constructors	Self-documenting method calls
Optional parameters tricky	Just skip them
Immutability	Final object built at the end
Maintainability	Easy to add new fields

6. When to Use Builder

✓ Use Builder when: - Object has **many optional parameters** - Object should be **immutable** - Step-by-step construction is required (e.g., SQL queries, documents, UI components) - Complex hierarchies/configurations

✗ Don't use Builder when: - Object is **simple** (few parameters) - Performance is critical (avoiding builder overhead)

7. Builder Analogy: Building a House

- **Client (Architect)** → Says what house is needed
- **Builder (Contractor)** → Provides steps (`addGarage`, `addPool`)
- **Concrete Builder (Workers)** → Builds foundation, walls, pool
- **Product (House)** → Final constructed house
- **Director (Project Manager)** → Decides build order

Builder vs Factory Comparison

Factory Pattern (Ready-made House)

```
class House {
    private String type;
    public House(String type) { this.type = type; }
    @Override public String toString() { return "House Type: " + type; }
}

class HouseFactory {
    public static House getHouse(String type) {
        switch (type) {
            case "1BHK": return new House("1BHK Flat");
            case "2BHK": return new House("2BHK Flat");
            case "Villa": return new House("Villa");
            default: throw new IllegalArgumentException("Unknown house
type");
        }
    }
}

public class FactoryDemo {
    public static void main(String[] args) {
        House h1 = HouseFactory.getHouse("1BHK");
        House h2 = HouseFactory.getHouse("Villa");
        System.out.println(h1);
        System.out.println(h2);
    }
}
```

Output:

```
House Type: 1BHK Flat
House Type: Villa
```

 Pick from predefined house types.

Builder Pattern (Custom House)

```
class CustomHouse {
    private int floors;
    private boolean hasGarage;
    private boolean hasPool;
    private boolean hasGarden;

    private CustomHouse(Builder b) {
        this.floors = b.floors;
    }
}
```

```

        this.hasGarage = b.hasGarage;
        this.hasPool = b.hasPool;
        this.hasGarden = b.hasGarden;
    }

    public static class Builder {
        private int floors;
        private boolean hasGarage;
        private boolean hasPool;
        private boolean hasGarden;

        public Builder floors(int f) { this.floors = f; return this; }
        public Builder garage(boolean g) { this.hasGarage = g; return this; }
        public Builder pool(boolean p) { this.hasPool = p; return this; }
        public Builder garden(boolean g) { this.hasGarden = g; return this; }

        public CustomHouse build() { return new CustomHouse(this); }
    }

    @Override
    public String toString() {
        return "CustomHouse [floors=" + floors +
            ", garage=" + hasGarage +
            ", pool=" + hasPool +
            ", garden=" + hasGarden + "]";
    }
}

public class BuilderDemo {
    public static void main(String[] args) {
        CustomHouse h = new CustomHouse.Builder()
            .floors(2)
            .garage(true)
            .pool(true)
            .garden(false)
            .build();

        System.out.println(h);
    }
}

```

Output:

```
CustomHouse [floors=2, garage=true, pool=true, garden=false]
```

 Build step by step, fully customizable.

Side-by-side Comparison

Feature	Factory Pattern	Builder Pattern
Goal	Choose <i>which type</i> of object	Define <i>how to build</i> a complex object
Analogy	Buy ready-made flat	Design your custom house
Flexibility	Low	High
Object Type	Predefined	Complex, customizable
When to Use	Standard objects (1BHK, 2BHK)	Complex customizable objects (custom house)
Java Examples	<code>Calendar.getInstance()</code>	<code>StringBuilder</code> , <code>ProcessBuilder</code>

✓ Summary:

- **Factory = Ready-made product (which object to create).**
- **Builder = Custom-made product (how to build it step by step).**